SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 - Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192424383
Student Name	A Yuvan Charan

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers 8 execute 10 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
  FOR j ← i + 1 TO n − 1 DO
    IF Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller THEN
      MarkDuplicate(j)
    END IF
  END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
  FOR j ← 0 TO i − 1 DO
    Audit(Request[j])        // basic operation
  END FOR
  i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing the key with the middle entry

and discarding half the index after each comparison; its comparison count satisfies $T(n) = T(n/2) + 1$, $T(1) = 1$.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size $n/2$ (because of replicated partitions) and merges the partial results in linear time: $T(n) = 4T(n/2) + n$. A proposed redesign removes the replication, giving $T(n) = 2T(n/2) + n$.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n + 10n \log n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's $\Theta(n \log n)$ claim is correct and give the tight bound. Verify your M2 result for $n = 1{,}024$.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for $n = 1{,}000{,}000$ and $p = 0.5$.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n + 10n \log n + 50$ is $\Theta(n \log n)$ by finding constants c_1, c_2 and n_0, and show that $f(n)$ is not $O(n)$.

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for $n = 1{,}000{,}000$. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach

Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20				
Application of Concepts	30				
Problem-Solving Approach	20				
Accuracy of Solution	20				
Clarity	10				
Total Marks					

Code of Conduct

I A YUVAN CHARAN (192424383), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

A Yuvan Charan

Date: 12 October 2026

(A) UNDERSTAND THE PROBLEM: MODULE BREAKDOWN

Notation: log = log ; L = log n. Server speed = 10^8 basic operations per second, so a 10-minute window (600 s) allows at most 6×10 basic operations. Today n = 10 ; next year n = 10 .

Module	Input size	Basic operation and where it occurs	Expected growth and justification
M1 Duplicate-Listing Detector	n = number of products	The test title = title AND seller = seller, in the IF of the inner loop; executed once for every pair (i, j) with j > i.	Pairs = n(n−1)/2, so quadratic growth: doubling n gives about 4× the work. MarkDuplicate runs at most once per pair and cannot dominate.
M2 API Rate Limiter	n = size of the largest request window	Audit(Request[j]) in the body of the inner FOR loop.	The inner loop runs 1, 2, 4, … times, a geometric series whose sum is about n: linear growth. The inner loop does NOT run n times in every pass.
M3 Product Search (S1, S2)	n = number of catalogue / index entries	Comparison of the key with an entry: S1 – each step of the sequential scan; S2 – each probe of the middle entry.	S1 examines up to n entries (linear). S2 halves the range per comparison, so about log n comparisons (logarithmic).
M4 Recommendation Engine	n = size of custo mer–product data	Handling one record in the linear merge step (the f(n) = n term), executed at every node of the recursion tree.	Current: 4 subproblems per level, so level work n, 2n, 4n, … and the leaves dominate – quadratic. Redesign: n per level over log n levels – n log n.
M5 Sales Analytics Engine	n = number of sales records	Record comparison in the sort done at each recursion node (n log n at the top); for the report generator, the arithmetic operations counted by f(n).	Recurrence: level cost n(L − level) over L levels gives n $\log^2$ n growth. f(n) = $3n^2$ + … is dominated by $3n^2$: quadratic.

(B) ANALYSE THE ITERATIVE MODULES

Module M1: Exact comparisons and complexities

For a fixed i the inner loop runs j = i+1 … n−1, i.e. (n − 1 − i) times. There is no early exit, so every pair is compared whatever the data:

$$C(n) = \Sigma_{i=0}^{n-2} (n - 1 - i) = (n-1) + (n-2) + \ldots + 2 + 1 = \mathbf{n(n-1)/2} = (n^2 - n)/2$$

Check: n = 4 gives 3 + 2 + 1 = 6 = 4·3/2. For n = 10^6, C = 499,999,500,000 ≈ 5 × 10^{11}.

Time complexity: Θ(n^2) in best, average and worst case. **Space complexity:** Θ(1) auxiliary (only i and j are used).

Module M2: Exact Audit operations and the team member's claim

The outer variable takes the values i = 2^k for k = 0, 1, 2, … while 2^k < n, i.e. k = 0 … ⌈log n⌉ − 1. Pass k runs the inner loop 2^k times:

$$A(n) = \Sigma_{k=0}^{\lceil \log n \rceil - 1} 2^k = 2^{\lceil \log n \rceil} - 1 \text{ (geometric series with ratio 2)}$$

If n is a power of two this is exactly **n − 1**. For any n, n ≤ $2^{\lceil \log n \rceil}$ < 2n, so n − 1 ≤ A(n) < 2n − 1. Hence **A(n) = Θ(n)**. Space = Θ(1).

The Θ(n log n) claim is incorrect. It multiplies the number of passes (about log n) by the largest inner-loop count (n). But the inner loop runs i times, and i equals n/2 only in the last pass, n/4 in the one before, and so on – half of all the work is done in the last pass. n log n is a valid but loose upper bound; it is not a lower bound because A(n) < 2n. The tight bound is Θ(n).

Verification for n = 1,024 = 2^{10} : the passes are i = 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 (i = 1024 fails i < n).

A(1024) = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512 = 2^{10} − 1 = **1,023** (n log n would give 1024 × 10 = 10,240)

(C) SOLVE A RECURRENCE BY SUBSTITUTION

Search version S2: $T(n) = T(n/2) + 1$, $T(1) = 1$. Take $n = 2^k$ and substitute backwards:

$T(n) = T(n/2) + 1$

$T(n) = [T(n/4) + 1] + 1 = T(n/2^2) + 2$

$T(n) = [T(n/8) + 1] + 2 = T(n/2^3) + 3$

…

$T(n) = T(n/2^i) + i$ (after i substitutions) The base case is reached when $n/2^i = 1$,

i.e. $i = \log n$. Substituting i and $T(1) = 1$:

$$T(n) = T(1) + \log_2 n = \mathbf{\log_2 n + 1}$$

Induction check: $T(1) = \log_2 1 + 1 = 1$ (true). If $T(n/2) = \log(n/2) + 1 = \log n$, then $T(n) = \log n + 1$ (true). For general n the closed form is $\lfloor \log_2 n \rfloor + 1$; for $n = 10^6$ this is 20 comparisons.

Implementation	Time	Space	Reason
Iterative binary search	$\Theta(\log n)$	$\Theta(1)$	A loop with low, high and mid; one comparison per iteration, about log n iterations; no extra memory.
Recursive binary search	$\Theta(\log n)$	$\Theta(\log n)$	Same comparisons, but each of the about log n nested calls keeps a stack frame (about 20 frames for $n = 10^6$).

(D) APPLY THE MASTER THEOREM

For $T(n) = aT(n/b) + f(n)$, compare f(n) with $n^{(\log_b a)}$. Case 1: $f(n) = O(n^{(\log_b a - \varepsilon)})$ gives $T(n) = \Theta(n^{(\log_b a)})$. Case 2: $f(n) = \Theta(n^{(\log_b a)})$ gives $T(n) = \Theta(n^{(\log_b a)} \cdot \log n)$. Case 3: $f(n) = \Omega(n^{(\log_b a + \varepsilon)})$ and $a \cdot f(n/b) \leq c \cdot f(n)$ for some $c < 1$ gives $T(n) = \Theta(f(n))$.

Module M4: Recommendation Engine

	Current design: $T(n) = 4T(n/2) + n$	**Proposed redesign: $T(n) = 2T(n/2) + n$**
a, b, f(n)	$a = 4, b = 2, f(n) = n$	$a = 2, b = 2, f(n) = n$
$n^{(\log_b a)}$	$n^{(\log 4)} = n^2$	$n^{(\log 2)} = n$
Comparison	$f(n) = n = O(n^{2-\varepsilon})$ with $\varepsilon = 1$ (f is polynomially smaller)	$f(n) = n = \Theta(n)$ (same order)
Case used	Case 1	Case 2
Result	$T(n) = \Theta(n^2)$	$T(n) = \Theta(n \log n)$
Exact solution, $T(1) = 1$	$T(n) = 2n^2 - n$. Check: $4[2(n/2)^2 - n/2] + n = 2n^2 - n$.	$T(n) = n \log n + n$. Check: $2[(n/2)(\log n - 1) + n/2] + n = n \log n + n$.
When the data doubles ($n \to 2n$)	$T(2n)/T(n) = (8n^2 - 2n)/(2n^2 - n) \to 4$. The running time quadruples.	$T(2n)/T(n) = 2(L + 2)/(L + 1) = 2 + 2/(L + 1)$, about 2.1 for $n = 10^6$. The time slightly more than doubles.

Meaning: with the current design a 10× growth in data means about 100× the work, and every doubling quadruples the nightly time. The redesign scales almost linearly, so it is the only version that survives the expansion.

Module M5: $T(n) = 2T(n/2) + n \log n$

Here $a = 2$, $b = 2$, $n^{(\log_b a)} = n$ and $f(n) = n \log n$. Case 1 fails (f is larger than n). Case 2 fails ($f \neq \Theta(n)$). Case 3 fails: $f(n)/n = \log n$, which is not $\Omega(n^{\varepsilon})$ for any $\varepsilon > 0$, so f is larger than n only by a log factor and not polynomially larger. **No standard case applies.** The extended Master Theorem ($f(n) = \Theta(n^{(\log_b a)} \cdot \log^k n)$ with $k \geq 0$ gives $T(n) = \Theta(n^{(\log_b a)} \cdot \log^{k+1} n)$) applies with $k = 1$:

$T(n) = \Theta(n \log^2 n)$

Recursion-tree derivation. Level j (j = 0 … L − 1) has 2^j nodes of size $n/2^j$, each costing $(n/2^j)\cdot\log(n/2^j)$:

Cost of level $j = 2^j \cdot (n/2^j) \cdot (L - j) = n(L - j)$

$T(n) = \Sigma_{j=0}^{L-1} n(L - j) + n\cdot T(1) = n[L + (L-1) + \ldots + 1] + n\cdot T(1) = n \cdot L(L + 1)/2 + n\cdot T(1)$

$= \Theta(n \log n)$

(E) ANALYSE BEST, AVERAGE AND WORST CASES FOR SEARCH

Case	S1 – sequential scan (unsorted)	S2 – binary search (sorted index)
Best	Key is the first entry: 1 comparison.	Key is the middle entry: 1 comparison.
Worst	Key is the last entry or is absent: n comparisons.	Key is found at the last probe (a leaf) or is absent: $\lfloor \log n \rfloor + 1$ comparisons. 2
Average	$A(n) = n - p(n - 1)/2$ (derived below).	Successful search $\approx \log n - 1$ comparisons; 2 unsuccessful $\approx \log n$. Hence $\Theta(\log n)$. 2

Average case of S1 with existence probability p

If the product exists (probability p), it is equally likely to be at any position 1 … n (probability 1/n each), so the expected cost is $\Sigma\, k\cdot(1/n) = (n + 1)/2$. If it does not exist (probability 1 − p), all n entries are examined. $A(n) = p \cdot (n + 1)/2 + (1 - p) \cdot n = n - p(n - 1)/2$ n = 1,000,000 and p = 0.5: A = 0.5 × 500,000.5 + 0.5 × 1,000,000 = 250,000.25 + 500,000 = **750,000.25 comparisons**

Check: p = 1 gives (n + 1)/2 and p = 0 gives n. At 10^8 operations per second this average is about 7.5 ms per lookup, whereas S2 needs at most 20 comparisons.

(F) PROVE AN ASYMPTOTIC BOUND

Definition: $f(n) = \Theta(n^2)$ if there exist constants $c_1, c_2 > 0$ and n_0 such that $c_1 n^2 \le f(n) \le c_2 n^2$ for all $n \ge n_0$.

Lower bound. For $n \ge 1$, $10n \log n \ge 0$ and $50 > 0$, so $f(n) \ge 3n^2$. Take **$c_1 = 3$**.

Upper bound. We show $10n \log n + 50 \le n^2$ for $n \ge 64$. At n = 64: $10 \cdot 64 \cdot 6 + 50 = 3{,}890 \le 4{,}096$. The function $g(n) = n^2 - 10n \log n - 50$ is increasing for $n \ge 64$ because $g'(n) = 2n - 10 \log n - 10/\ln 2 > 0$ there, so the inequality holds for all $n \ge 64$. Then $f(n) \le 3n^2 + n^2 = 4n^2$. Take **$c_2 = 4$** and **$n_0 = 64$**.

$3n^2 \le 3n^2 + 10n \log n + 50 \le 4n^2$ for all $n \ge 64$, therefore **$f(n) = \Theta(n^2)$**. 2 (A cruder choice also works: using $\log n \le n$ and $50 \le 50n^2$ gives c = 63 and $n_0 = 1$.) Cross-check: $\lim f(n)/n^2 = 3$, 2 0 a finite positive constant. **f(n) is not O(n).**

Suppose $f(n) \le C\cdot n$ for all $n \ge n_1$, for some constant C > 0. Since $f(n) \ge 3n^2$, this requires $3n^2 \le C\cdot n$, i.e. $n \le C/3$, for every $n \ge n_1$. Choosing $n = \max(n_1, \lceil C \rceil + 1) > C/3$ gives a contradiction. Equivalently, $f(n)/n = 3n + 10 \log n + 50/n \to \infty$. Hence f(n) is not O(n).

(G) SCALABILITY DECISION (n = 1,000,000)

Server capacity = 10^8 operations/second. Nightly window = 10 minutes = 600 s. Maximum allowed operations = $600 \times 10^8 = 6 \times 10^{10}$.

Module	Operations at $n = 10^6$	Running time at 10^8 ops/s	Today ($n = 10^5$)	Meets 10-min window?
M1: $n(n - 1)/2$	$\approx 5.0 \times 10^{11}$	≈ 5,000 s ≈ 83.3 min	$5 \times 10^9 \to 50$ s	FAIL (8.3× over)
M2: $2^{\lceil \log n \rceil} - 1$	$1{,}048{,}575 \approx 1.05 \times 10^6$	≈ 0.0105 s	131,071 → 1.3 ms	PASS

S2: $\lfloor \log n \rfloor + 1$ (per search)	20	≈ 0.2 µs	17	PASS
M4 current: $2n^2 - n$	$\approx 2.0 \times 10^{12}$	$\approx 20{,}000$ s ≈ 333 min	$2 \times 10^{10} \rightarrow 200$ s	FAIL (33× over)
M4 redesign: $n \log n + n$	$\approx 2.09 \times 10^{7}$	≈ 0.21 s	$1.76 \times 10^{6} \rightarrow 17.6$ ms	PASS
M5 recurrence: $n \cdot L(L+1)/2 + n$	$\approx 2.10 \times 10^{8}$	≈ 2.1 s	$1.47 \times 10^{7} \rightarrow 0.15$ s	PASS
M5 report generator: $f(n)$	$\approx 3.0002 \times 10^{12}$	$\approx 30{,}002$ s ≈ 500 min	$3.0 \times 10^{10} \rightarrow 300$ s	FAIL (50× over)

Observation: every module fits the window today (the slowest needs 300 s), but a 10× growth multiplies each quadratic module by 100× and pushes it far beyond 600 s. M2, S2, the redesigned M4 and the M5 recurrence need no change.

Modules that must be redesigned

1. **M1 Duplicate-Listing Detector ($\Theta(n^2)$, 83 min).** Replace the pairwise comparison with a hash set keyed on (title, seller): make one pass, and mark a product as a duplicate if its key is already present. Expected time $\Theta(n) \approx 10^6$ operations (about 0.01 s), extra space $\Theta(n)$. Alternatively, sort by (title, seller) in $\Theta(n \log n) \approx 2 \times 10^7$ operations and compare only neighbours. Checking each new upload against the hash index would remove the nightly batch altogether.
2. **M4 Recommendation Engine, current design ($\Theta(n^2)$, 333 min).** Adopt the proposed redesign. Removing the replicated partitions changes $4T(n/2) + n$ into $2T(n/2) + n$, i.e. $\Theta(n^2)$ into $\Theta(n \log n)$: about 10^5 times fewer operations (2×10^{12} to 2.1×10^7) and a run time of 0.21 s.
3. **M5 nightly report generator ($\Theta(n^2)$, about 500 min).** The $3n^2$ term must be removed by building the report from pre-aggregated data: keep running totals per product, seller and day in indexed tables updated per sale, or use sort-and-scan / hash group-by. This gives $\Theta(n \log n)$, about $10 \cdot n \cdot \log n \approx 2 \times 10^8$ operations ≈ 2 s. Adding servers alone would give only a constant-factor gain (about 50× is needed), so an algorithmic fix is preferred.

Conclusion: redesign M1, M4 (use the n log n version) and the M5 report generator before the catalogue reaches 10^6 products. Keep M2 ($\Theta(n)$), S2 ($\Theta(\log n)$) and the M5 sort recurrence ($\Theta(n \log^2 n)$), and retire S1 in favour of S2 (about 750,000 average comparisons versus about 20).